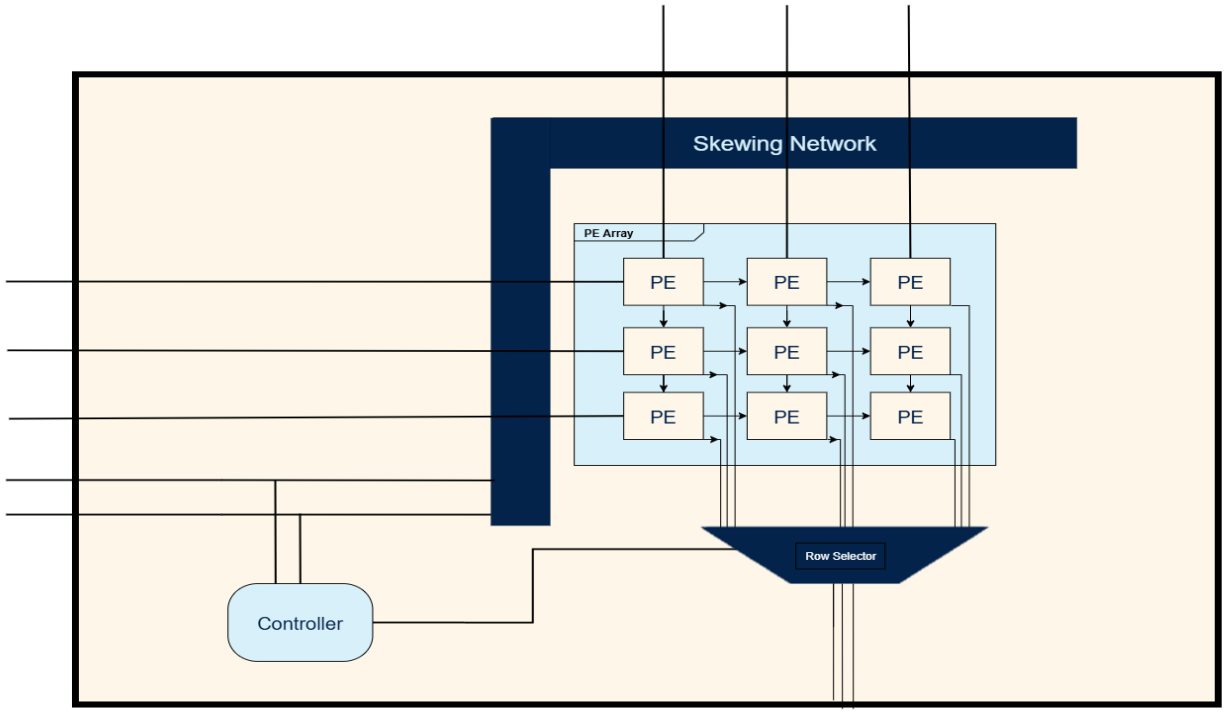
Systolic Array Matrix Multiplier

Table of Contents

1. Architecture
2. Module Declarations
3. Simulation and Verification
    a. Testbench Setup
    b. Test Case 1: Basic Matrix Multiplication
    c. Test Case 2: Equal Matrix Test
    d. Test Case 3: Identity Matrix Test
    e. Test Case 4: Zero Matrix Test
4. Simulation Tools

1. Architecture



The systolic array implements matrix multiplication C = A × B using an NxN grid of processing elements. The system operates in four coordinated phases:

1. **Input Phase**: Matrix A columns and B rows are fed with progressive skewing
2. **Computation Phase**: Data flows through PEs performing multiply-accumulate operations
3. **Output Phase**: Results are extracted row by row
4. **Clear Phase**: PE accumulators are reset for next computation

Key components work together: input skewing network ensures proper timing alignment, PE array performs parallel computation, and FSM controller orchestrates the operation phases.

---

2. Module Declarations

2.0 Type Definitions

```
typedef logic [DATAWIDTH-1:0]   data_word_t;
typedef logic [2*DATAWIDTH-1:0] result_word_t;
```

Type Definitions Explanation:

- `data_word_t`: Represents input matrix element data with configurable width (DATAWIDTH bits). Chosen for clean parameterization and readability across all modules.

- `result_word_t`: Represents multiplication result with double width (2*DATAWIDTH bits) to prevent overflow during multiply-accumulate operations. Essential for maintaining precision in matrix computations.

2.1 Top-Level Module: `systolic_array`

```
module systolic_array #(

    parameter int DATAWIDTH = 16,
    parameter int N_SIZE    = 5

)(
    input  logic            clk,
    input  logic            rst_n,
    input  logic            valid_in,
    input  data_word_t      matrix_a_in  [N_SIZE]
    input  data_word_t      matrix_b_in  [N_SIZE],
    output logic            valid_out,
    output result_word_t    matrix_c_out [N_SIZE]

);
```

The top-level systolic array module serves as the main coordinator for matrix multiplication operations. It accepts matrix A data as column vectors and matrix B data as row vectors through unpacked arrays. The module instantiates four key submodules: the FSM controller for timing coordination, input skewing network for data alignment, PE array for computation, and row selector for output multiplexing. The parameterized design allows scalable implementation from small 3×3 arrays for verification to larger arrays for high-performance applications.

2.2 Controller: `systolic_controller`

```
module systolic_controller #(

    parameter int N_SIZE = 5

)(

    input  logic                        clk,

    input  logic                        rst_n,

    input  logic                        valid_in,

    output logic                        valid_out,

    output logic [$clog2(N_SIZE)-1:0]   row_sel,

    output logic                        feed_new_data,

    output logic                        clear_accumulator

);
```

IMPLEMENTATION DETAILS - STRUCTURED APPROACH:

The systolic controller implements a **4-state finite state machine** with **2 dedicated counters** for precise timing control:

Counter Implementation:

- `cycle_counter`: Global operation timing (`logic [$clog2(2*N_SIZE+1)-1:0]`)
- `input_counter`: Input data stream counting (`logic [$clog2(N_SIZE+1)-1:0]`)

FSM States (STRUCTURED DESIGN):

```
typedef enum logic [2:0] {
    IDLE,    // Waiting for input data (reset counters)
    INPUT,   // Accepting input stream (count valid inputs)
    COMPUTE, // Generating output results (count total cycles)
    DONE     // Clear accumulators (single cycle transition)
} fsm_state_t;
```

State Transitions:

- IDLE → INPUT: When `valid_in` asserted
- INPUT → COMPUTE: When `input_counter >= N_SIZE`
- COMPUTE → DONE: When `cycle_counter >= 2*N_SIZE-1`
- DONE → IDLE: Automatic (single cycle clear)

Control Signal Generation:

- `feed_new_data = valid_in` - Input gating control (allows true systolic behavior)
- `clear_accumulator = (current_state == DONE)` - PE reset control
- `valid_out = output_window_active` - Result validity timing
- `row_sel = cycle_counter - N_SIZE - 1` - Output row selection

Key Timing: Results available from cycle N_SIZE+1 to 2*N_SIZE, enabling pipelined operation.

The controller orchestrates the entire matrix multiplication process through these precisely timed phases, ensuring proper data flow and result extraction.

2.3 Input Network: `input_skewing_network`

```
module input_skewing_network #(
    parameter int DATAWIDTH = 16,
    parameter int N_SIZE    = 5
)(
```

```
    input  logic        clk,
    input  logic        rst_n,
    input  logic        feed_new_data,
    input  data_word_t  a_columns_in  [N_SIZE],
    input  data_word_t  b_rows_in     [N_SIZE],
    output data_word_t  a_columns_out [N_SIZE],
    output data_word_t  b_rows_out    [N_SIZE]
);
```

The input skewing network creates the essential timing alignment required for proper systolic operation. It implements progressive delay chains where data element `i` receives `i` clock cycles of delay, creating a diagonal wavefront of data arrival across the PE array. This ensures all processing elements begin computation simultaneously despite staggered input timing.

Generate Block Implementation:

```
generate
    for (genvar i = 0; i < N_SIZE; i++) begin : skewing_elements
        // Element 0 has no delay, element i has i cycles of delay
        if (i == 0) begin
            assign a_columns_out[0] = gated_a_input;
            assign b_rows_out[0]    = gated_b_input;
        end else begin
            shift_register #(.WIDTH(DATAWIDTH), .STAGES(i)) a_delay_chain (...);
            shift_register #(.WIDTH(DATAWIDTH), .STAGES(i)) b_delay_chain (...);
        end
    end
endgenerate
```

The generate block enables scalable hardware generation where each iteration creates delay chains of increasing length. The conditional logic handles the special case of element 0 (direct connection) while instantiating parametric shift registers for delayed elements. Input gating through `feed_new_data` prevents spurious data during computation phases by injecting zeros when new data feeding is disabled.

2.4 PE Array: `processing_element_array`

```
module processing_element_array #(
    parameter int DATAWIDTH = 16,
```

```
    parameter int N_SIZE    = 5

)(

    input  logic            clk,

    input  logic            rst_n,

    input  logic            clear_accumulator,

    input  data_word_t      row_data       [N_SIZE],

    input  data_word_t      col_data       [N_SIZE],

    output result_word_t    pe_results_out [N_SIZE][N_SIZE]

);
```

The processing element array forms the computational heart using **structured generate blocks** (NOT flattened approach):

Generate Block Implementation:

```
generate

    for (genvar row = 0; row < N_SIZE; row++) begin : pe_row_generation
        for (genvar col = 0; col < N_SIZE; col++) begin : pe_col_generation
            processing_element #(.DATAWIDTH(DATAWIDTH)) pe_instance (

                .clk              (clk),

                .rst_n            (rst_n),

                .clear_accumulator(clear_accumulator),

                .a_in             (horizontal_bus[row][col]),     // From left neighbor

                .b_in             (vertical_bus[row][col]),       // From top neighbor

                .a_out            (horizontal_bus[row][col+1]),   // To right neighbor

                .b_out            (vertical_bus[row+1][col]),     // To bottom neighbor

                .sum_out          (pe_results_out[row][col])      // Accumulated result

            );

        end
    end
endgenerate
```

IMPLEMENTATION DETAILS:

- **Systematic Generation:** N^2 PEs created via nested generate loops
- **Clear Signal Distribution:** `clear_accumulator` propagated to all PEs

- **Mesh Interconnection:** Horizontal and vertical systolic data busses
- **Modular Structure:** Each PE instance independently instantiated

2.5 Processing Element: `processing_element`

```
module processing_element #(

    parameter int DATAWIDTH = 16

)(

    input  logic            clk,

    input  logic            rst_n,

    input  logic            clear_accumulator,

    input  data_word_t      a_in,

    input  data_word_t      b_in,

    output data_word_t      a_out,

    output data_word_t      b_out,

    output result_word_t    sum_out

);
```

IMPLEMENTATION DETAILS:

Each processing element implements multiply-accumulate with **clear accumulator control**:

Internal Implementation:

- **3 Registers:** `a_register`, `b_register`, `accumulator`
- **Combinational Logic:** `product = a_in * b_in`
- **Conditional Accumulation:**

```
if (clear_accumulator)
      accumulator <= '0;
else
      accumulator <= accumulator + product;
```

Clear Feature: The `clear_accumulator` signal enables resetting PE accumulators during the DONE state, ensuring clean computation cycles. This provides precise control over when accumulation begins and ends, essential for back-to-back matrix operations.

2.6 Shift Register: `shift_register`

```
module shift_register #(

    parameter int WIDTH  = 16,

    parameter int STAGES = 1
```

```
)(
    input  logic        clk,
    input  logic        rst_n,
    input  data_word_t  data_in,
    output data_word_t  data_out
);
```

The shift register provides configurable delay elements essential for input skewing. Each instance creates a delay chain of STAGES clock cycles, enabling the progressive timing alignment required for systolic operation.

2.7 Row Selector: `row_selector`

```
module row_selector #(
    parameter int DATAWIDTH = 16,
    parameter int N_SIZE    = 5
)(
    input  logic [$clog2(N_SIZE)-1:0]   sel,
    input  result_word_t                data_in [N_SIZE][N_SIZE],
    output result_word_t                data_out[N_SIZE]
);
```

The row selector module implements a parallel-to-serial output multiplexer that extracts complete matrix rows from the PE array results. It receives the full N×N result matrix from all processing elements and uses the controller's `row_sel` signal to determine which row to output during each cycle. This enables sequential streaming of result rows while the controller sequences through indices 0 to N-1. The module includes bounds checking to prevent invalid selections and supports the pipelined result extraction essential for continuous systolic operation.

---

3. Simulation and Verification

3.1 Testbench Setup

The testbench uses 3×3 matrices with 2-bit data width for simple verification. It includes comprehensive monitoring capabilities with cycle-by-cycle PE state debugging and automatic result validation.

Configuration: `DATAWIDTH = 2`, `N_SIZE = 3`, `CLK_PERIOD = 10ns`

Test Tasks:

- `run_basic_multiplication_test()` - Basic matrix multiplication verification
- `equal_matrix_test()` - Uniform matrix values test
- `run_identity_matrix_test()` - Identity matrix multiplication test
- `run_zero_matrix_test()` - Zero matrix edge case test

Helper Tasks:

- `reset_dut()` - Initializes DUT to known state with proper reset sequence
- `feed_matrix_data()` - Feeds input matrices with proper timing using negedge setup
- `capture_results()` - Waits for and captures all output rows with validation
- `check_results()` - Compares actual vs expected results with detailed error reporting
- `debug_pe_array_state()` - Displays PE internal states for debugging
- `start_pe_monitoring()` - Enables continuous cycle-by-cycle PE monitoring
- `print_final_summary()` - Generates comprehensive test report with pass/fail status

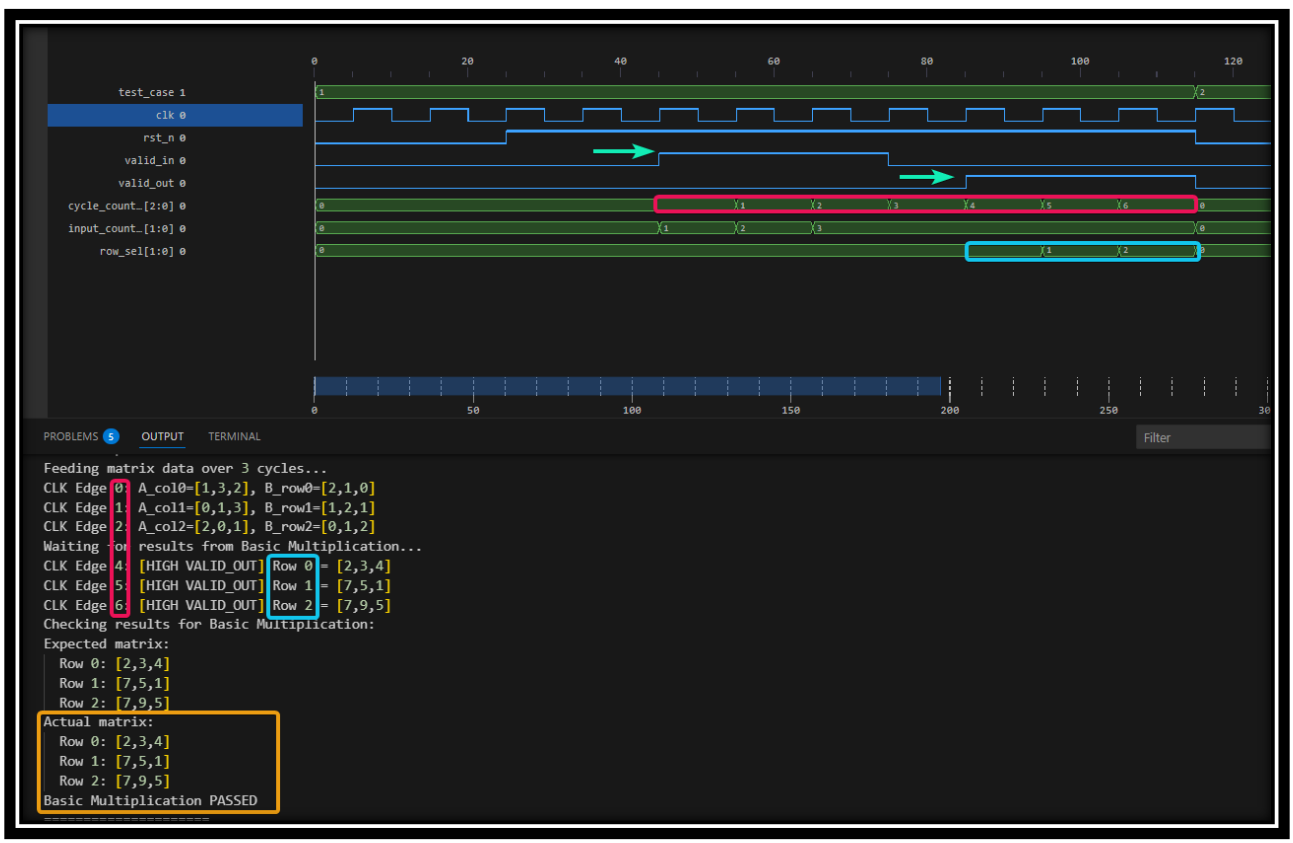
3.2 Test Case 1: Basic Matrix Multiplication

Purpose: Verify correct matrix multiplication with non-trivial values.

Test Matrices:

```
Matrix A = [1 0 2]    Matrix B = [2 1 0]

           [3 1 0]               [1 2 1]

           [2 3 1]               [0 1 2]

Expected C = [2 3 4]

             [7 5 1]

             [7 9 5]
```

Computation Proof: C[0][1] = 1×1 + 0×2 + 2×1 = 3, C[1][0] = 3×2 + 1×1 + 0×0 = 7



3.3 Test Case 2: Equal Matrix Test

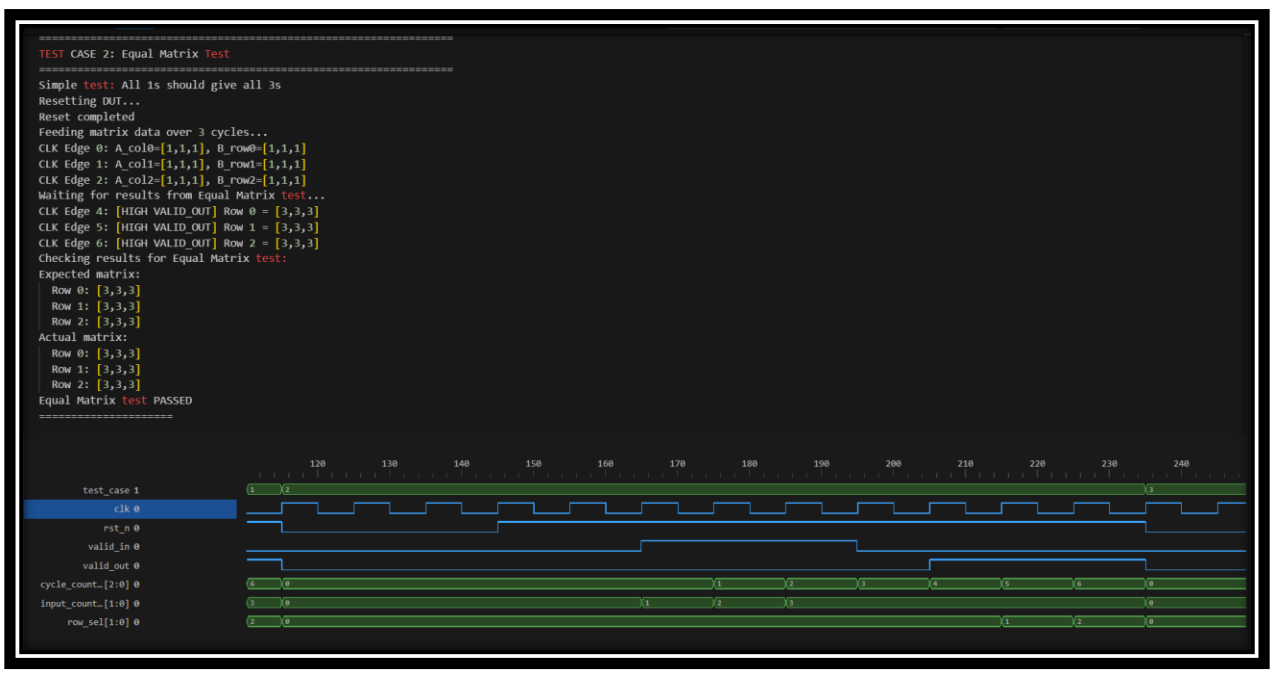
Purpose: Test with uniform matrix values to verify accumulation logic.

Test Matrices:

```
Matrix A = [1 1 1]    Matrix B = [1 1 1]

           [1 1 1]               [1 1 1]

           [1 1 1]               [1 1 1]


Expected C = [3 3 3]

             [3 3 3]

             [3 3 3]
```

Computation Proof: Each element C[i][j] = 1×1 + 1×1 + 1×1 = 3

Result appeared at cycle 4 ,5 and 6 as expected with valid out also being asserted in that range



3.4 Test Case 3: Identity Matrix Test

Purpose: Verify multiplication with identity matrices.

Test Matrices:

```
Matrix A = [1 0 0]    Matrix B = [1 0 0]
           [0 1 0]               [0 1 0]
           [0 0 1]               [0 0 1]

Expected C = [1 0 0]
             [0 1 0]
             [0 0 1]
```

Computation Proof: Identity property I×I = I, e.g., C[0][0] = 1×1 + 0×0 + 0×0 = 1

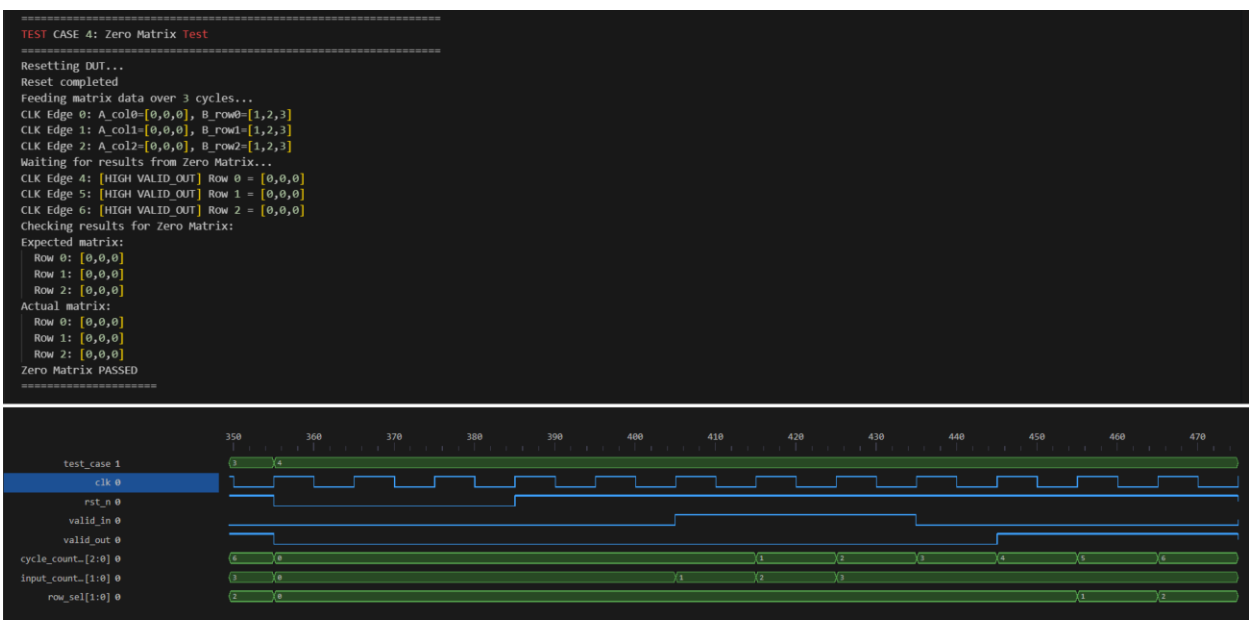
3.5 Test Case 4: Zero Matrix Test

Purpose: Test edge case with zero matrix multiplication.

Test Matrices:

```
Matrix A = [0 0 0]    Matrix B = [1 2 3]

           [0 0 0]               [1 2 3]

           [0 0 0]               [1 2 3]


Expected C = [0 0 0]

             [0 0 0]

             [0 0 0]
```

Computation Proof: Zero property 0×B = 0, e.g., C[0][0] = 0×1 + 0×1 + 0×1 = 0



4. Simulation Tools

The design was successfully verified using multiple simulation environments:

- **Local Simulator**: Dsim Studio - Used for primary development and debugging
- **Online Platform**: EDA Playground with 4 commercial simulators:
  - Cadence Xcelium
  - Synopsys VCS
  - Mentor Questa
  - Aldec Riviera-PRO
  - All simulators showed consistent results
  - Cross-platform compatibility confirmed

Live Demo: Interactive demonstration available at EDA Playground - Ready-to-run environment with complete testbench. Simply press "Run" to see the systolic array in action with all test cases.

All simulators were used with free licenses. All test cases passed across different simulation tools, demonstrating robust design implementation.